Unit 2
Line, Circle, and Polygon.

2.1 Image Representation

Point is the fundamental element of the picture representation. It is nothing but the position in a plane defined as either pairs or triplets of numbers depending on whether the data are two or three dimensional. Thus, (x_1, y_1) or (x_1, y_1, z_1) would represent a point in either two or three dimensional space. Two points would represent a line or edge, and a collection of three or more points a polygon. The representation of curved lines is usually accomplished by approximating them by short straight line segments.

If the two points used to specify a line are (x_1, y_1) and (x_2, y_2), then an equation for the line is given as

$$\frac{y-y_1}{x-x_1} = \frac{y_2-y_1}{x_2-x_1} \quad \ldots (2.1)$$

$$\therefore \quad y = \frac{y_2-y_1}{x_2-x_1}(x-x_1) + y_1 \quad \ldots(2.2)$$

or

$$y = mx + b \quad \ldots(2.3)$$

where

$$m = \frac{y_2-y_1}{x_2-x_1} \quad \ldots(2.4)$$

and

$$b = y_1 - mx_1 \quad \ldots(2.5)$$

The above equation is called the **slope intercept form** of the line. The **slope** m is the change in height $(y_2 - y_1)$ divided by the change in width $(x_2 - x_1)$ for two points on the line. The **intercept** b is the height at which the line crosses the y-axis.

Another form of the line equation, called the general form, may be found by multiplying out the factors and rearranging the equation (2.1).

$$(y_2 - y_1)\,X - (x_2 - x_1)\,y + x_2\ \ y_1 - x_1\,y_2 = 0 \quad \ldots(2.6)$$

or

$$rx + sy + t = 0 \quad \ldots (2.7)$$

2.3.1 Digital Differential Analyzer

We know that the slope of a straight line is given as

$$m = \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1} \quad \ldots (2.1)$$

The above differential equation can be used to obtain a rasterized straight line. For any given x interval Δx along a line, we can compute the corresponding y interval Δy from equation 2.1 as

$$\Delta y = \frac{y_2 - y_1}{x_2 - x_1} \Delta x \quad \ldots (2.2)$$

Similarly, we can obtain the x interval Δx corresponding to a specified Δy as

$$\Delta x = \frac{x_2 - x_1}{y_2 - y_1} \Delta y \quad \ldots (2.3)$$

Once the intervals are known the values for next x and next y on the straight line can be obtained as follows

$$x_{i+1} = x_i + \Delta x = x_i + \frac{x_2 - x_1}{y_2 - y_1} \Delta y \quad \ldots (2.4)$$

and

$$y_{i+1} = y_i + \Delta y = y_i + \frac{y_2 - y_1}{x_2 - x_1} \Delta x \quad \ldots (2.5)$$

The equations 2.4 and 2.5 represent a recursion relation for successive values of x and y along the required line. Such a way of rasterizing a line is called a **digital differential analyzer** (DDA). For simple DDA either Δx or Δy, whichever is larger, is chosen as one raster unit, i.e.

if $|\Delta x| \geq |\Delta y|$ then

differential analyzer (DDA). For simple DDA either Δx or Δy, whichever is larger, is chosen as one raster unit, i.e.

if $|\Delta x| \geq |\Delta y|$ **then**

else

$$\Delta x = 1$$

$$\Delta y = 1$$

With this simplification, if $\Delta x = 1$ then

we have $y_{i+1} = y_i + \frac{y_2 - y_1}{x_2 - x_1}$ **and**

$$x_{i+1} = x_i + 1$$

If $\Delta y = 1$ then

we have

$$y_{i+1} = y_i + 1 \text{ and}$$

$$x_{i+1} = x_i + \frac{x_2 - x_1}{y_2 - y_1}$$

DDA line Drawing Algorithm

1. Read the line end points (x_1, y_1) and (x_2, y_2) such that they are not equal.

   [if equal then plot that point and exit]

2. $\Delta x = |x_2 - x_1|$ and $\Delta y = |y_2 - y_1|$

3. if $(\Delta x \geq \Delta y)$ then

   length = Δx

   else

   length = Δy

   end if

4. $\Delta x = (x_2 - x_1) / \text{length}$

   $\Delta y = (y_2 - y_1) / \text{length}$

   [This makes either Δx or Δy equal to 1 because length is either $|x_2 - x_1|$ or $|y_2 - y_1|$. Therefore, the incremental value for either x or y is one.]

5. $x = x_1 + 0.5 * \text{Sign}(\Delta x)$

   $y = y_1 + 0.5 * \text{Sign}(\Delta y)$

   [Here, Sign function makes the algorithm work in all quadrant. It returns $-1, 0, 1$ depending on whether its argument is < 0, $= 0$, > 0 respectively. The factor 0.5 makes it possible to round the values in the integer function rather than truncating them.]

6. i = 1

   [Begins the loop, in this loop points are plotted]

   While (i ≤ length)

   {

   Plot (Integer (x), Integer (y))

   x = x + Δx

   y = y + Δy

   i = i + 1

   }

7. Stop

Let us see few examples to illustrate this algorithm.

Example 2.1 : *Consider the line from (0, 0) to (4, 6). Use the simple DDA algorithm to rasterize this line.*

Solution : Evaluating steps 1 to 5 in the DDA algorithm we have,

$x_1 = 0 \qquad y_1 = 0 \qquad x_2 = 4 \qquad y_2 = 6$

$\therefore \quad \text{Length} = |y_2 - y_1| = 6$

$\therefore \quad \Delta x = |x_2 - x_1| \,/\, \text{length}$

$= \frac{4}{6}$

and $\quad \Delta y = |y_2 - y_1| \,/\, \text{length}$

$= 6 \,/\, 6 = 1$

Initial value for

$$x = 0 + 0.5 * \text{Sign}\left(\frac{4}{6}\right) = 0.5$$

$$y = 0 + 0.5 * \text{Sign}\,(1) = 0.5$$

Tabulating the results of each iteration in the step 6 we get,

i	Plot	x	y
		0.5	0.5
1	(0, 0)		
		1.167	1.5
2	(1, 1)		
		1.833	2.5
3	(1, 2)		
		2.5	3.5
4	(2, 3)		
		3.167	4.5
5	(3, 4)		
		3.833	5.5
6	(3, 5)		
		4.5	6.5

The results are plotted as shown in the Fig. 2.5. It shows that the rasterized line lies to both sides of the actual line, i.e. the algorithm is **orientation dependent.**

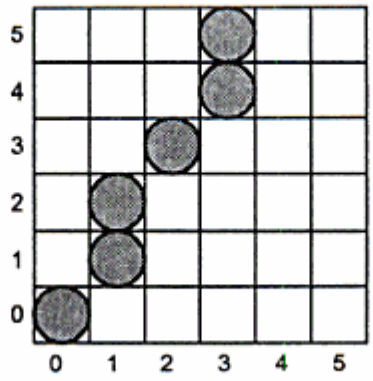


Fig. 2.5 Result for a simple DDA

➠ **Example 2.2 :** *Consider the line from (0, 0) to (– 6, – 6). Use the simple DDA algorithm to rasterize this line.*

Advantages of DDA Algorithm

1. It is the simplest algorithm and it does not require special skills for implementation.
2. It is a faster method for calculating pixel positions than the direct use of equation $y = mx + b$. It eliminates the multiplication in the equation by making use of raster characteristics, so that appropriate increments are applied in the x or y direction to find the pixel positions along the line path.

Disadvantages of DDA Algorithm

1. Floating point arithmetic in DDA algorithm is still time-consuming.
2. The algorithm is orientation dependent. Hence end point accuracy is poor.

2.2.2 Bresenham's Line Algorithm

Bresenham's line algorithm uses only integer addition and subtraction and multiplication by 2, and we know that the computer can perform the operations of integer addition and subtraction very rapidly. The computer is also time-efficient when performing integer multiplication by powers of 2. Therefore, it is an efficient method for scan-converting straight lines.

The basic principle of Bresenham's line algorithm is to select the optimum raster locations to represent a straight line. To accomplish this the algorithm always increments either x or y by one unit depending on the slope of line. The increment in the other variable is determined by examining the distance between the actual line location and the nearest pixel. This distance is called **decision variable** or the **error**. This is illustrated in the Fig. 2.7.

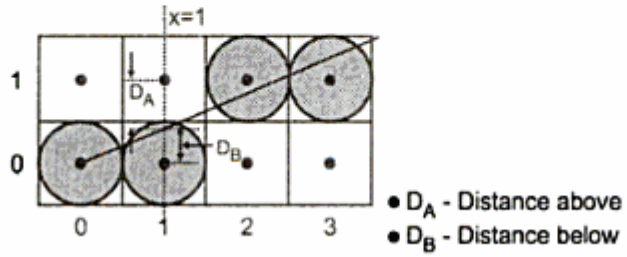


Fig. 2.7

As shown in the Fig. 2.7, the line does not pass through all raster points (pixels). It passes through raster point (0, 0) and subsequently crosses three pixels. It is seen that the intercept of line with the line x = 1 is closer to the line y = 0, i.e. pixel (1, 0) than to the line y = 1 i.e. pixel (1, 1). Hence, in this case, the raster point at (1, 0) better represents the path of the line than that at (1, 1). The intercept of the line with the line x = 2 is close to the line y = 1, i.e. pixel (2, 1) than to the line y = 0, i.e. pixel (2, 0). Hence, the raster point at (2, 1) better represents the path of the line, as shown in the Fig. 2.7.

In mathematical terms error or decision variable is defined as,

$$e = D_B - D_A \quad \text{or} \quad D_A - D_B$$

Let us define $e = D_B - D_A$. Now if $e > 0$, then it implies that $D_B > D_A$, i.e., the pixel above the line is closer to the true line. If $D_B < D_A$ (i.e. $e < 0$) then we can say that the pixel below the line is closer to the true line. Thus by checking only the sign of error term it is possible to determine the better pixel to represent the line path.

The error term is initially set as

$$e = 2\,\Delta y - \Delta x$$

where $\Delta y = y_2 - y_1$, and $\Delta x = x_2 - x_1$

Then according to value of e following actions are taken.

```
while ( e ≥ 0)
    {
        y = y + 1
        e = e – 2 * Δx
    }
x = x + 1
e = e + 2 * Δy
```

When $e \geq 0$, error is initialized with $e = e - 2\,\Delta x$. This is continued till error is negative. In each iteration y is incremented by 1. When $e < 0$, error is initialized to $e = e + 2\,\Delta y$. In both the cases x is incremented by 1. Let us see the Bresenham's line drawing algorithm.

Bresenham's Line Algorithm

1. Read the line end points (x_1, y_1) and (x_2, y_2) such that they are not equal.
   [if equal then plot that point and exit]
2. $\Delta x = |x_2 - x_1|$ and $\Delta y = |y_2 - y_1|$
3. [Initialize starting point]
   $x = x_1$
   $y = y_1$
4. $e = 2 * \Delta y - \Delta x$
   [Initialize value of decision variable or error to compensate for nonzero intercepts]
5. i = 1 [Initialize counter]
6. Plot (x, y)
7. while (e ≥ 0)
   {
   y = y + 1
   e = e – 2 * Δx
   }
   x = x + 1
   e = e + 2 * Δy
8. i = i + 1
9. if (i ≤ Δx) then go to step 6.
10. Stop

Example 2.3 : *Consider the line from (5, 5) to (13, 9). Use the Bresenham's algorithm to rasterize the line.*

Solution : Evaluating steps 1 through 4 in the Bresenham's algorithm we have,

$$\Delta x = | 13 - 5 | = 8$$

$$\Delta y = | 9 - 5 | = 4$$

$$x = 5$$

$$y = 5$$

$$e = 2 * \Delta y - \Delta x = 2 * 4 - 8$$

$$= 0$$

Tabulating the results of each iteration in the step 5 through 10.

i	Plot	x	y	e
		5	5	0
1	(5, 5)	6	6	– 8
2	(6, 6)	7	6	0
3	(7, 6)	8	7	– 8
4	(8, 7)	9	7	0
5	(9, 7)	10	8	– 8
6	(10, 8)	11	8	0
7	(11, 8)	12	9	– 8
8	(12, 9)	13	9	0
9	(13, 9)	14	10	– 8

Table 2.3

The results are plotted as shown in Fig. 2.8.

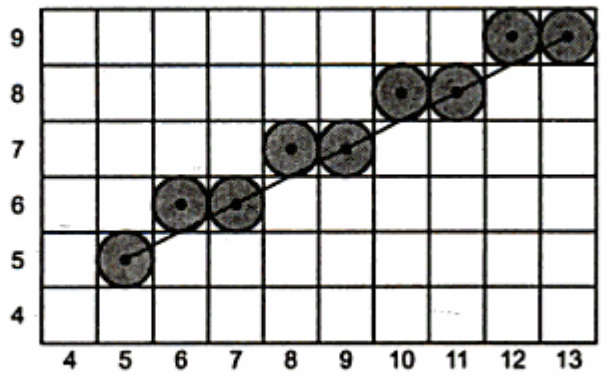


Fig. 2.8

2.7 Basic Concepts in Circle Drawing

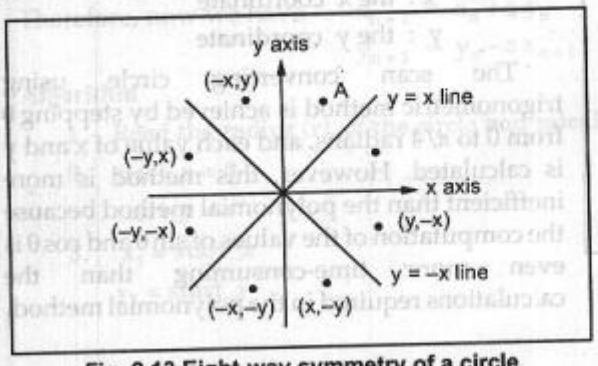


Fig. 2.13 Eight-way symmetry of a circle

A circle is a symmetrical figure. It has eight-way symmetry as shown in the Fig. 2.13. Thus, any circle generating algorithm can take advantage of the circle symmetry to plot eight points by calculating the coordinates of any one point. For example, if point A in the Fig. 2.13 is calculated with a circle algorithm, seven more points could be found just by reflection.

2.8 Representation of a Circle

There are two standard methods of mathematically representing a circle centered at the origin.

2.8.1 Polynomial Method

In this method circle is represented by a polynomial equation.

$$x^2 + y^2 = r^2$$

where
- x : the x coordinate
- y : the y coordinate
- r : radius of the circle

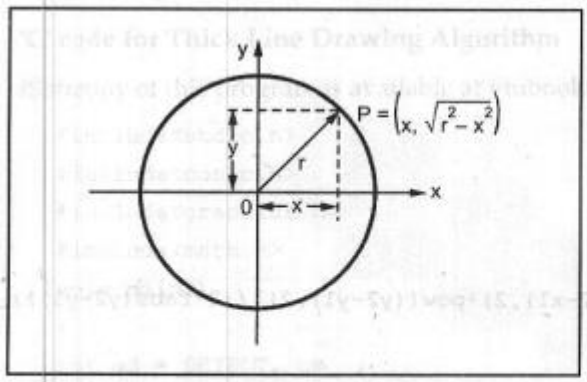
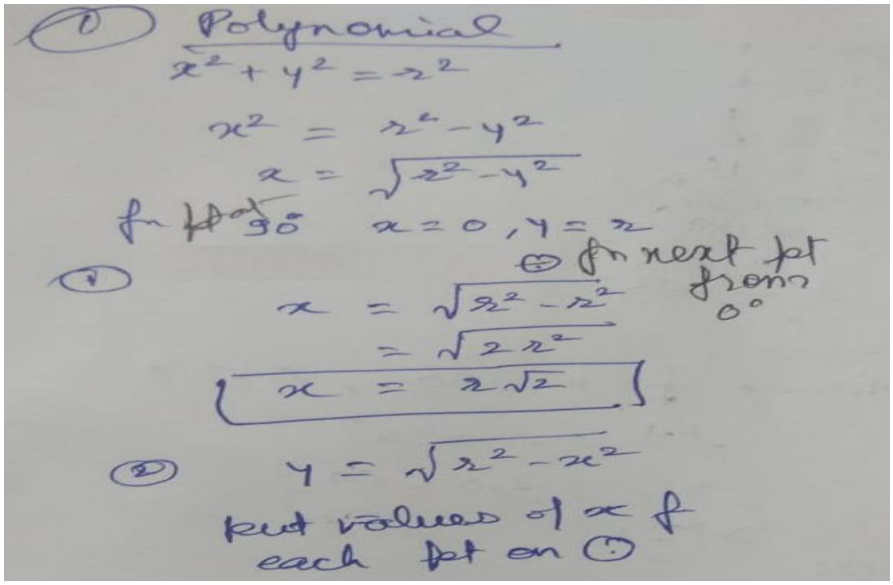
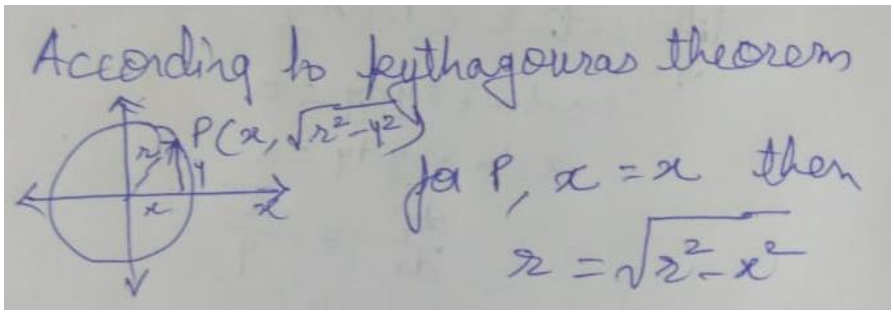


Fig. 2.14 Scan converting circle using polynomial method

Here, polynomial equation can be used to find y coordinate for the known x coordinate. Therefore, the scan converting circle using polynomial method is achieved by stepping x from 0 to $r\sqrt{2}$, and each y coordinate is found by evaluating $\sqrt{r^2 - x^2}$ for each step of x. This generates the 1/8 portion (90° to 45°) of the circle. Remaining part of the circle can be generated just by reflection.

The polynomial method of circle generation is an inefficient method. In this method for each point both x and r must be squared and x^2 must be subtracted from r^2; then the square root of the result must be found out.





2.8.2. Trigonometric Method

In this method, the circle is represented by use of trigonometric functions

$$x = r\cos\theta \quad \text{and} \quad y = r\sin\theta$$

where θ : current angle

r : radius of the circle

x : the x coordinate

y : the y coordinate

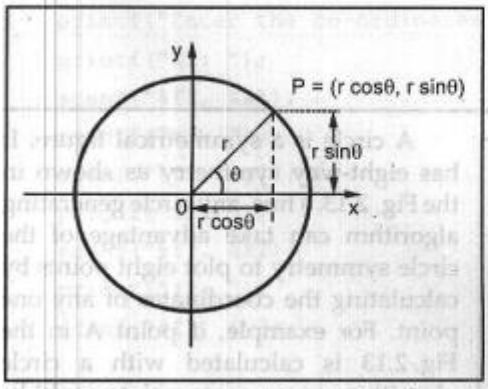


Fig. 2.15 Scan converting circle using trigonometric method

The scan converting circle using trigonometric method is achieved by stepping θ from 0 to $\pi/4$ radians, and each value of x and y is calculated. However, this method is more inefficient than the polynomial method because the computation of the values of $\sin\theta$ and $\cos\theta$ is even more time-consuming than the calculations required in the polynomial method.

2.9 Circle Drawing Algorithms

In this section we are going to discuss three efficient circle drawing algorithms :

- DDA algorithm and
- Bresenham's algorithm
- Midpoint algorithm

2.9.1 DDA Circle Drawing Algorithm

We know that, the equation of circle, with origin as the center of the circle is given as

$$x^2 + y^2 = r^2$$

The digital differential analyser algorithm can be used to draw the circle by defining circle as a differential equation. It is as given below

$$2x\,dx + 2y\,dy = 0 \quad \text{where } r \text{ is constant}$$

$$\therefore \quad x\,dx + y\,dy = 0$$

$$\therefore \quad y\,dy = -x\,dx$$

$$\therefore \quad \frac{dy}{dx} = \frac{-x}{y}$$

From above equation, we can construct the circle by using incremental x value, $\Delta x = \varepsilon y$ and incremental y value, $\Delta y = -\varepsilon x$, where ε is calculated from the radius of the circle as given below

$$2^{n-1} \le r < 2^n \quad r : \text{radius of the circle}$$

$$\varepsilon = 2^{-n}$$

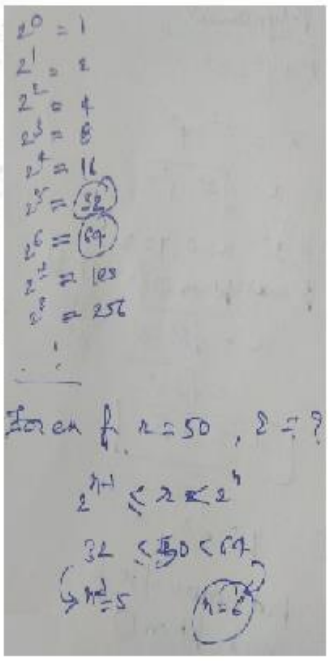
For example, if r = 50 then n = 6 so that $32 \le 50 < 64$

$$\therefore \quad \varepsilon = 2^{-6} = 0.0156$$

Applying these incremental steps we have,

$$x_{n+1} = x_n + \varepsilon y_n$$

$$y_{n+1} = y_n - \varepsilon x_n$$



Applying these incremental steps we have,

$$x_{n+1} = x_n + \varepsilon y_n$$

$$y_{n+1} = y_n - \varepsilon x_n$$

The points plotted using above equations give the spiral instead of the circle. To get the circle we have to make one correction in the equation; we have to replace x_n by x_{n+1} in the equation of y_{n+1}.

Therefore, now we have

$$x_{n+1} = x_n + \varepsilon y_n$$

$$y_{n+1} = y_n - \varepsilon x_{n+1}$$

Algorithm

1. Read the radius (r), of the circle and calculate value of ε
2. start_x = 0

   start_y = r
3. x_1 = start_x

   y_1 = start_y

4. do

   [$x_2 = x_1 + \varepsilon y_1$

   $y_2 = y_1 - \varepsilon x_2$

   [x_2 represents x_{n+1} and x_1 represents x_n]

   Plot (int (x_2) , int (y_2))

   $x_1 = x_2$;

   $y_1 = y_2$;

   [Reinitialize the current point]

   } while (y_1 – start_y) < ε or (start_x – x_1) > ε

   [check if the current point is the starting point or not. If current point is not starting point repeat step 4 ; otherwise stop]
5. Stop.

2.6.2 Bresenham's Circle Drawing Algorithm

The Bresenham's circle drawing algorithm considers the eight-way symmetry of the circle to generate it. It plots $1/8^{th}$ part of the circle, i.e. from 90° to 45°, as shown in the Fig. 2.24. As circle is drawn from 90° to 45°, the x moves in positive direction and y moves in the negative direction.

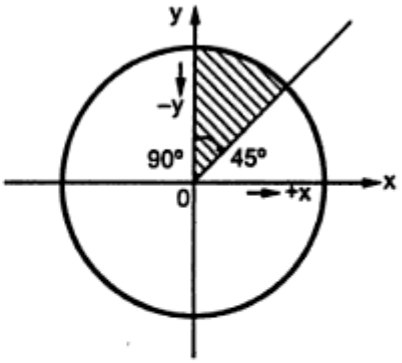


Fig. 2.24 1/8 part of circle

To achieve best approximation to the true circle we have to select those pixels in the raster that fall the least distance from the true circle. Refer Fig. 2.25. Let us observe the 90° to 45° portion of the circle. It can be noticed that if points are generated from 90° to 45°, each new point closest to the true circle can be found by applying either of the two options :

- Increment in positive x direction by one unit or
- Increment in positive x direction and negative y direction both by one unit.

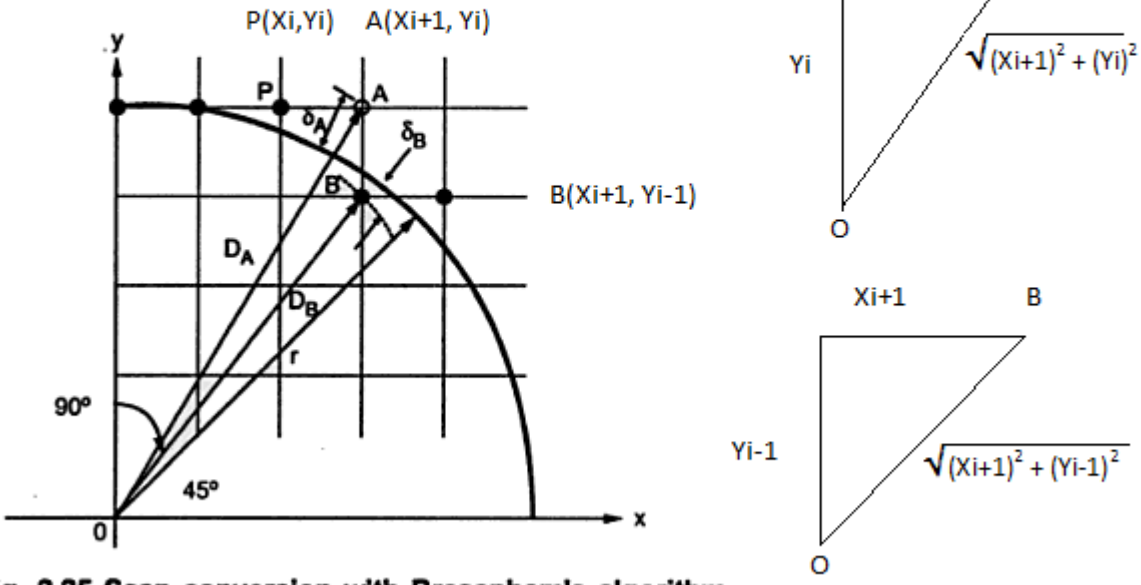


Fig. 2.25 Scan conversion with Bresenham's algorithm

Let us assume point P in Fig. 2.25 as a last scan converted pixel. Now we have two options either to choose pixel A or pixel B. The closer pixel amongst these two can be determined as follows.

The distances of pixels A and B from the origin are given as,

$$D_A = \sqrt{(x_{i+1})^2 + (y_i)^2} \quad \text{and}$$

$$D_B = \sqrt{(x_{i+1})^2 + (y_i - 1)^2}$$

Now, the distances of pixels A and B from the true circle are given as,

$$\delta_A = D_A - r \text{ and } \delta_B = D_B - r$$

However, to avoid square root term in derivation of decision variable, i.e. to simplify the computation and to make algorithm more efficient the δ_A and δ_B are defined as,

$$\delta_A = D_A^2 - r^2 \quad \text{and}$$

$$\delta_B = D_B^2 - r^2$$

From Fig. 2.25, we can observe that δ_A is always positive and δ_B always negative. Therefore, we can define **decision variable** d_i as

$$d_i = \delta_A + \delta_B$$

and we can say that, if $d_i < 0$, i.e., $\delta_A < \delta_B$ then only x is incremented; otherwise x is incremented in positive direction and y is incremented in negative direction. In other words we can write,

For $d_i < 0$, $x_{i+1} = x_i + 1$ and

For $d_i \geq 0$, $x_{i+1} = x_i + 1$ and $y_{i+1} = y_i - 1$

The equation for d_i at starting point, i.e. at $x = 0$ and $y = r$ can be simplified as follows,

$$\begin{aligned} d_i &= \delta_A + \delta_B \\ &= (x_i + 1)^2 + (y_i)^2 - r^2 + (x_i + 1)^2 + (y_i - 1)^2 - r^2 \\ &= (0 + 1)^2 + (r)^2 - r^2 + (0 + 1)^2 + (r - 1)^2 - r^2 \\ &= 1 + r^2 - r^2 + 1 + r^2 - 2r + 1 - r^2 \\ &= 3 - 2r \end{aligned}$$

Similarly, the equations for d_{i+1} for both the cases are given as,

For $d_i < 0$, $d_{i+1} = d_i + 4x_i + 6$ and

For $d_i \leq 0$, $d_{i+1} = d_i + 4(x_i - y_i) + 10$

Algorithm to plot 1/8 of the circle

1. Read the radius (r) of the circle.
2. d = 3 – 2 r
   [Initialize the decision variable]
3. x = 0, y = r
   [Initialize starting point]
4. do
   {
   plot (x, y)
   if (d < 0) then
   {

   {
   d = d + 4x + 6
   }
   else
   { d = d + 4 (x – y) + 10
   y = y – 1
   }
   x = x + 1
   } while (x < y)
5. Stop

The remaining part of circle can be drawn by reflecting point about y axis, x axis and about origin as shown in Fig. 2.18.

Therefore, by adding seven more plot commands after the plot command in the step 4 of the algorithm, the circle can be plotted. The remaining seven plot commands are :

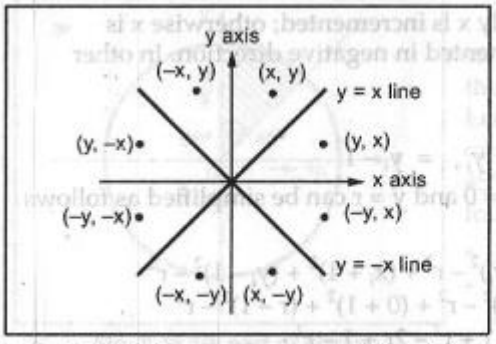


Fig. 2.18 Eight-way symmetry of the circle

plot (y, x)
plot (y, – x)
plot (x, – y)
plot (– x, – y)
plot (– y, – x)
plot (– y, x) and
plot (– x, y)

2.6.3 Midpoint Circle Drawing Algorithm

The midpoint circle drawing algorithm also uses the eight-way symmetry of the circle to generate it. It plots 1/8 part of the circle, i.e. from 90° to 45°, as shown in the Fig. 2.27. As circle is drawn from 90 to 45°, the x moves in the positive direction and y moves in the negative direction. To draw a 1/8 part of a circle we take unit steps in the positive x direction and make use of decision parameter to determine which of the two possible y positions is closer to the circle path at each step. The Fig. 2.27 shows the two possible y positions (y_i and $y_i + 1$) at sampling position $x_i + 1$. Therefore, we have to determine whether the pixel at position $(x_i + 1, y_i)$ or at position $(x_i + 1, y_i - 1)$ is closer to the circle. For this purpose decision parameter is used. It uses the circle function ($f_{circle}(x, y) = x^2 + y^2 - r^2$) evaluated at the midpoint between these two pixels.

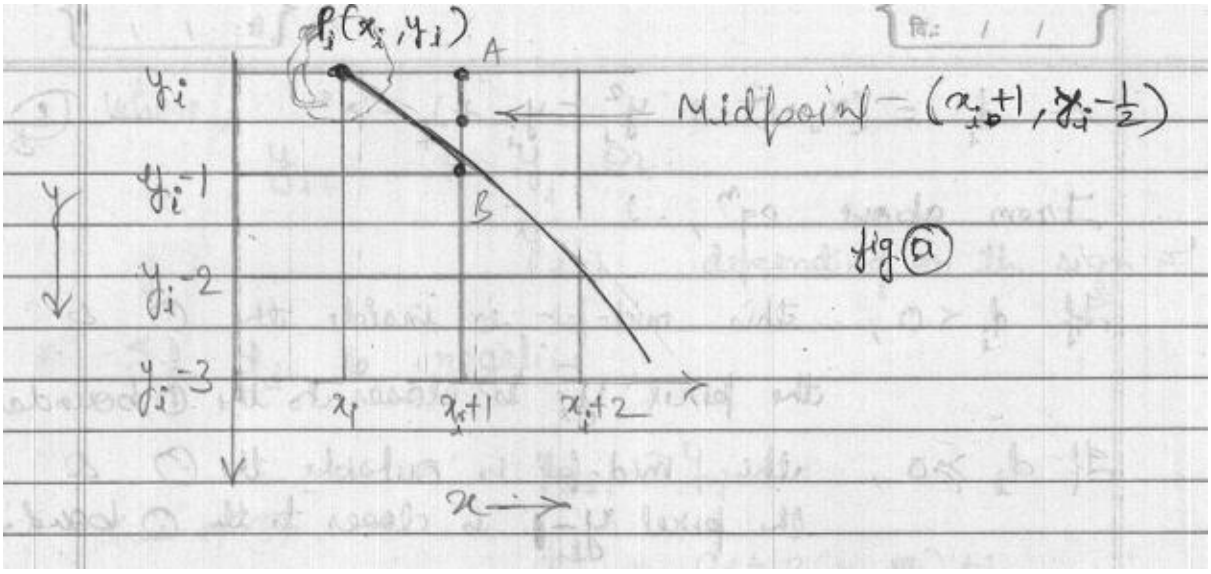


Fig. 2.27 Decision parameter to select correct pixel in circle generation algorithm

From above fig ⓐ point (x_i, y_i) is plotted (known pt. is plotted)

Now next, algorithm determines whether the pixel at position $(x_i + 1, y_i) = A$ or pixel at position $(x_i + 1, y_i - 1) = B$ is near to circle boundary and plot pixel whichever is near.

For this, decision parameter is ~~used~~ evaluated at the mid-point of these two pixels.

From fig ⓐ, co-ordinates of the mid-pt are

$$\left(x_i + 1, y_i - \frac{1}{2}\right)$$

It uses the circle f^n.

$$f_{circle}(x, y) = x^2 + y^2 - r^2.$$

putting values of midpt co-ordinates.

$$d_i = f_{circle}(x, y)$$

$$d_i = (x_i + 1)^2 + (y_i - \tfrac{1}{2})^2 - r^2 \quad \text{— eqⁿ (3ⓐ)}$$

$$\begin{aligned} d_i &= f_{circle}\left(x_i + 1, y_i - \frac{1}{2}\right) \\ &= (x_i + 1)^2 + \left(y_i - \frac{1}{2}\right)^2 - r^2 \\ &= (x_i + 1)^2 + y_i^2 - y_i + \frac{1}{4} - r^2 \end{aligned} \quad \text{... (2.30)}$$

If $d_i < 0$, this midpoint is inside the circle and the pixel on the scan line y_i is closer to the circle boundary. If $d_i \geq 0$, the midposition is outside or on the circle boundary, and $y_i - 1$ is closer to the circle boundary. The incremental calculation can be determined to obtain the successive decision parameters. We can obtain a recursive expression for the next decision parameter by evaluating the circle function at sampling position $x_{i+1} + 1 = x_i + 2$.

$$d_{i+1} = f_{circle}\left(x_{i+1}+1, y_{i+1}-\frac{1}{2}\right)$$

$$= \left[(x_i+1)+1\right]^2+\left(y_{i+1}-\frac{1}{2}\right)^2-r^2$$

$$= (x_i+1)^2+2(x_i+1)+1+y_{i+1}^2-\left(y_{i+1}\right)+\frac{1}{4}-r^2 \quad \ldots (2.31)$$

Looking at equations (2.30) and (2.31) we can write

$$d_{i+1} = d_i + 2(x_i+1) + \left(y_{i+1}^2-y_i^2\right)-\left(y_{i+1}-y_i\right)+1$$

where y_{i+1} is either y_i or y_{i-1}, depending on the sign of d_i.

If d_i is negative, $y_{i+1} = y_i$

$\therefore$ $$d_{i+1} = d_i + 2(x_i+1) + 1$$

$$= d_i + 2x_{i+1} + 1 \quad \ldots (2.32)$$

If d_i is positive, $y_{i+1} = y_{i-1}$

$\therefore$ $$d_{i+1} = d_i + 2(x_i+1) + 1 - 2y_{i+1} \quad \ldots (2.33)$$

The terms $2x_{i+1}$ and $-2y_{i+1}$ in equations (2.8) and (2.9) can be incrementally calculated as

$$2x_{i+1} = 2x_i + 2$$

$$2y_{i+1} = 2y_i - 2$$

The initial value of decision parameter can be obtained by evaluating circle function at the start position $(x_0, y_0) = (0, r)$.

$$d_0 = f_{circle}\left((0+1)^2+\left(r-\frac{1}{2}\right)^2-r^2\right)$$

$$= 1+\left(r-\frac{1}{2}\right)^2-r^2$$

$$= 1.25 - r$$

Algorithm

1. Read the radius (r) of the circle
2. Initialize starting position as

   x = 0

   y = r
3. Calculate initial value of decision parameter as

   d = 1.25 – r
4. do

```
{       plot (x, y)

        if (d < 0)
                {       x = x + 1
                        y = y
                        d = d + 2x + 1
                }
        else
                {       x = x + 1
                        y = y – 1
                        d = d + 2x + 2y + 1
        }
                while (x < y)
```

5. Determine symmetry points
6. Stop.